Assignment 3: Revenue of Intercollegiate Athletic Programs in the USA

FEM11149 - Introduction to Data Science

Olivia Windisch

October, 2026

1. Introduction

Intercollegiate athletic programs are an important part of higher education in the United States, but their revenues differ greatly: while some larger athletic programs may earn over $100 million USD per year, many programs at smaller colleges have less than $1 million USD available for their teams. For college athletic directors, it is therefore useful to know what revenue can be expected for a program with certain characteristics, for budget planning and for comparisons with similar institutions. Institutions that receive federal student aid report information about their athletic programs to the US Department of Education (USDE), for example about athletes, coaches and coach salaries. As many of these characteristics are closely related, it is relevant to investigate whether this information can be used to estimate the revenue of a program, and whether a few general features of a program already contain as much information as all characteristics together.

Using data on 1,000 colleges and universities, this report answers two research questions. First, *to what extent can the revenue of an intercollegiate athletic program be predicted from a few underlying program characteristics, compared with using all characteristics separately?* Secondly, *which characteristics, either on their own or combined into these underlying features, are most closely related to revenue?*

2. Data

The data come from the Equity in Athletics Disclosure Act (EADA) survey of the USDE. Every college or university with federal student aid and intercollegiate athletics reports these data annually (USDE, 2026). The dataset used for this report contains 1,000 institutions. Two variables are categorical: the classification of the athletic association and division (19 levels, e.g. NCAA Division I-FBS) and the sector of the institution (5 levels, e.g. private nonprofit). The other 28 predictor variables describe undergraduate enrollment, the number of athletes and teams, the participants per sport, the number and average salaries of head and assistant coaches, and the shares of full-time and female head coaches, mostly separately for men and women. The unit ID is not used as a predictor. The variable of interest is the total revenue of the athletic program, which includes for example ticket sales, media rights, donations and support from the institution. Since revenue has to cover the expenses of the program, a deficit paid by the institution also counts as revenue (USDE, 2025). As revenue is very right-skewed, the logarithm of revenue, which is much more symmetric (Appendix, Figure 1), is used as the dependent variable.

To evaluate the predictions on unseen data, the institutions are randomly split into a training set of 800 (80%) and a test set of 200 institutions (20%).

Missing values only occur in the coach variables. Institutions without men's or women's teams report no coaches for those teams, so the number of coaches and their salaries are set to zero. For 10 institutions, assistant coaches are reported without a salary (16 missing salary values in total). These may be volunteers,

who are left out of the average salary (USDE, 2026), but as this cannot be verified, they are replaced by the median of the training data. The skewed counts and salaries are transformed with $\log(1+x)$, which keeps zeros defined and reduces the influence of very large programs.

A second dataset contains 10 institutions (IDs 1 to 10) for which the revenue is predicted.

3. Methods

To reduce the number of predictors, *Principal Component Analysis (PCA)* is used. This unsupervised technique describes the variables as well as possible by a few new, uncorrelated variables, the principal components, which are weighted sums of the original variables. This is useful when many variables are strongly correlated and therefore measure more or less the same thing.

Let $\mathbf{X}$ be the $n \times p$ matrix of predictors $X_1, \dots, X_p$, with n observations and p predictors. Each principal component is a linear combination of the predictors, $Z_s = \gamma_{1s}X_1 + \gamma_{2s}X_2 + \cdots + \gamma_{ps}X_p = \gamma_s^\top \mathbf{X}$, where $\gamma_s = (\gamma_{1s}, \dots, \gamma_{ps})^\top$ is the loading vector with the weights of component s. The first component has the largest possible variance:

$$\max_{\gamma_1 : \|\gamma_1\|=1} \mathrm{Var}(\gamma_1^\top \mathbf{X}) = \max_{\gamma_1 : \|\gamma_1\|=1} \gamma_1^\top \Sigma \gamma_1,$$

where $\Sigma = \mathrm{Var}(\mathbf{X})$ is the covariance matrix of the predictors. The restriction $\|\gamma_1\| = 1$ prevents an infinitely large variance. The solution is the first eigenvector of Σ and $\mathrm{Var}(Z_1)$ is the first eigenvalue λ_1. Each next component again has maximum variance, but is uncorrelated with the previous components. As a result, higher components only describe the variance that is still left, which makes them harder to interpret. This procedure gives eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p$, where $\lambda_s = \mathrm{Var}(Z_s)$ measures the importance of component s. As PCA is sensitive to scale, it is done on the correlation matrix. The eigenvalues then add up to the number of variables, $\sum_{s=1}^{p} \lambda_s = p$, and the proportion of variance accounted for by component s is $\mathrm{VAF}_s = \lambda_s / p$.

Four different methods are then used to select a meaningful number of dimensions. First, the *cumulative VAF* should be at least 80%, although this can lead to many components. Second, *Kaiser's rule* keeps all components with $\lambda_s > 1$. Such a component explains more than one original variable, as a component equal to one standardized variable would have an eigenvalue of exactly 1. Third, in the *scree plot* of λ_s against s, components up to the "elbow" are kept (without the elbow itself). Lastly, a *permutation test* compares each observed eigenvalue λ_s to the eigenvalues of $N_{\text{perm}} = 1000$ datasets in which the values within each column are randomly permuted, which removes the correlations between the predictors. From the eigenvalues of the permuted datasets, the $\alpha/2$ and $1 - \alpha/2$ percentiles are computed (with $\alpha = 0.05$). If λ_s lies above this interval, dimension s is significantly different from random data and is kept.

The stability of the eigenvalues and VAF is checked with the *bootstrap*: $B = 1000$ bootstrap datasets of size $n \times p$ are drawn by sampling rows of $\mathbf{X}$ with replacement. The 2.5 and 97.5 percentiles of the eigenvalues give a 95% confidence interval. An eigenvalue is significantly larger than 1 if the whole interval lies above 1, and a narrow interval for the cumulative VAF indicates a stable result.

To interpret the components, the component loadings, i.e. the correlations between the original variables and the component scores, are used: $\rho_{js} = \mathrm{cor}(X_j, Z_s) = \gamma_{js}\lambda_s^{1/2}$. If a component has high correlations with a group of related variables, it can be given a name. A biplot shows the observations as points and the variables as arrows for the first two components. Finally, the fit per variable in r dimensions, $\sum_{s=1}^{r} \rho_{js}^2$, can be interpreted as the proportion of the variance of variable X_j that is accounted for by the first r components. A value close to 1 means that the variable is well represented.

In the principal component regression (PCR), the dependent variable is regressed with Ordinary Least Squares (OLS) on the first r component scores and dummy variables for the categorical predictors:

$$y_i = \beta_0 + \sum_{s=1}^{r} \theta_s z_{is} + \sum_{l=1}^{L} \delta_l d_{il} + \varepsilon_i,$$

where y_i is the dependent variable for observation i, β_0 the intercept, z_{is} the score of observation i on component s with coefficient θ_s, d_{il} the l-th of L dummy variables with coefficient δ_l, and ε_i the error term.

Whether a component is related to the dependent variable is tested with $t = \hat{\theta}_s / \mathrm{SE}(\hat{\theta}_s)$, the estimate divided by its standard error. The p-value is the probability of a t-value at least this large if the null hypothesis (no relation) is true; the component is significant at the 5% level if $p < 0.05$, or equivalently $|t| > 1.96$. A higher R^2 means that more variance of the dependent variable is explained by the model.

In addition to the PCR, a *ridge regression* is applied. This type of multiple regression penalizes the size of the coefficients, which stabilizes the estimates when there are many or strongly correlated predictors. Ridge regression minimizes

$$L(\beta_1, \ldots, \beta_q) = \underbrace{\sum_{i=1}^{n} \Big(y_i - \sum_{j=1}^{q} x_{ij} \beta_j \Big)^2}_{\text{regression term}} + \underbrace{\lambda \sum_{j=1}^{q} \beta_j^2}_{\text{penalty term}} ,$$

where q is the number of predictors including the dummy variables, x_{ij} the value of predictor j for observation i, β_j the regression weight of predictor j, and $\lambda > 0$ the penalty parameter (not the PCA eigenvalues). Here y_i is measured in deviation from its mean, so that the intercept is not penalized. Ridge regression shrinks the β_j towards zero and λ controls the strength of this shrinkage. As $\lambda \to 0$, the ridge estimates become the OLS estimates, and as $\lambda \to \infty$, they go towards zero. Since all weights are penalized equally, the numerical predictors are standardized to z-scores first. Ridge regression is preferred over LASSO since LASSO can set coefficients to zero and tends to keep only one of several strongly correlated predictors. Ridge regression keeps all of them.

A good value of λ is chosen with K-fold cross-validation with $K = 10$. Each fold k is left out once, and the model estimated on the other folds predicts its n_k observations, which gives

$$\mathrm{MSE}_k = \frac{1}{n_k} \sum_{i \in \text{fold } k} (y_i - \hat{y}_i)^2, \qquad \mathrm{RMSE}_{\mathrm{CV}} = \sqrt{\frac{1}{K} \sum_{k=1}^{K} \mathrm{MSE}_k},$$

where $\hat{y}_i$ is the out-of-sample prediction for observation i. Two values are considered: $\lambda_{\min}$, the value with the lowest $\mathrm{RMSE}_{\mathrm{CV}}$, and λ_{1SE}, the largest λ whose error is within one standard error of this minimum. The 1-SE rule gives a simpler model with more shrinkage.

Both models are estimated on the training set and compared on the test set with $\mathrm{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2}$ and $R^2 = 1 - \sum_i (y_i - \hat{y}_i)^2 / \sum_i (y_i - \bar{y})^2$, where m is the number of test observations and $\bar{y}$ their mean. A lower RMSE and a higher R^2 mean better predictions. The RMSE on the log scale is the main evaluation metric: errors on the log scale are relative errors, which suits athletic directors comparing programs of very different sizes, as a relative error of 20% then counts the same for a program with 1 million USD as for one with 100 million USD revenue. Squaring the errors penalizes large mistakes, which matter most for a budget. To check whether the errors depend on the size of a program, the test institutions are sorted by revenue and split into ten equally sized revenue groups (each containing 10% of the test institutions), and the test RMSE is computed per group. Predictions are transformed back to USD with $\exp(\hat{y}_i)$. The RMSE in USD is also reported, since budgets are planned in dollars, but on the original scale it is mostly determined by the largest observations.

4. Results

The first principal component has an eigenvalue of 13.80 and explains 49.3% of the variance of the 28 predictors (Appendix, Table 1). The selection methods do not agree: the 80% VAF rule suggests eight components, Kaiser's rule six, the bootstrapped Kaiser's rule five, the scree plot two, and the permutation test three. Since the permutation test is the only method that compares the eigenvalues to those of random data, three components are selected. Together, they explain 64.4% of the variance. The narrow bootstrap interval [63.1%, 66.5%] shows that this result is stable.

The loadings (Appendix, Table 2) and the biplot (Appendix, Figure 2) show what the components mean. The first component correlates positively with all variables, most strongly with the number of athletes, teams and coaches, so it measures *Program Size*. The second component, *Quality vs Quantity*, is high for programs

at large institutions with high coach salaries and many full-time coaches, and low for programs with many teams and coaches but lower salaries. The third component separates individual sports, such as track and field, from the team sports basketball, baseball and softball. This *Individual vs Team Sports* component is less clear than the first two components, because it is forced to be orthogonal to them: after *Program Size* and *Quality vs Quantity* have been accounted for, the third component can only describe the variation that is left, which does not necessarily form a meaningful direction. This is in line with its small VAF (5.4%). Football participation is hardly related to it (-0.04); football, the main revenue driver at FBS institutions, is mostly captured by the first component and by the division dummies in the PCR.

In the PCR, the components and dummies explain 94.5% of the variance of log revenue in the training data. *Program Size* and *Quality vs Quantity* are statistically significant at the 0.1% level ($p < 0.001$). *Individual vs Team Sports* is not statistically significant ($p = 0.64$). The division is also very important: an NCAA Division I-FBS program earns about 11 times as much as a CCCAA program, ceteris paribus.

For ridge regression, cross-validation chose a very small penalty (λ_{min} = 5.0e-04), so the results are close to OLS. The largest ridge coefficients are the division dummy variables, followed by the salary of head coaches of men's teams.

On the test set, both models predict log revenue well (Appendix, Table 3). The PCR has the lowest RMSE on the log scale (0.425, $R^2 = 0.916$), so three components and the divisions contain almost the same information as all 28 characteristics. On the original scale, ridge regression with λ_{min} has the lowest RMSE (8.78 million USD). These errors mainly come from the largest programs: the seven FBS programs in the test set cause 71% of the squared error. Per revenue group (Appendix, Table 4), the errors on the log scale are largest for the 20% smallest and the 10% largest programs (RMSE of about 0.5 to 0.7, compared with about 0.2 to 0.4 in the middle groups), and the largest programs are systematically underpredicted by about 16%. The predicted revenues of the ten new institutions (Appendix, Table 5) range from 1.7 million USD for the NAIA Division I program to 115.2 (PCR) and 142.2 million USD (ridge) for the FBS program.

5. Conclusions and Discussion

Regarding the first research question, revenue can be predicted quite well on the log scale. Three principal components, together with the athletic division, predict log revenue on the test set with an R^2 of 0.916. On the log scale, the PCR predictions are slightly better than the ridge regression with all 28 characteristics. A few underlying factors therefore contain almost all relevant information. Nevertheless, a prediction is typically off by a factor of about 1.5 (exp(0.425)), and the errors in USD are largest for the biggest programs. The models can be useful as a benchmark for athletic program directors, but should not be used for predicting the exact revenue of one program.

Regarding the second research question, revenue is mostly related to the size of a program, to a lesser extent to how well its coaches are paid, and strongly to the division: FBS programs earn much more than similar programs in lower divisions.

This research has several limitations. Most importantly, the results only show correlation, not causation. The data are self-reported and describe one point in time, and the imputed salaries may actually belong to unpaid volunteers. Moreover, components were formed without looking at revenue, so football, which is a crucial sport for the largest programs, is not captured well by them. The models were compared on only one train/test split, and the smallest and largest programs are predicted worst. This could be improved by allowing the effects to differ by division, for example with interaction terms or separate models for NCAA Division I and the other divisions, as the revenue of the largest programs is driven by football, media rights and conference distributions. Variables on these income sources (e.g. attendance or media contracts) and more very small and very large programs in the data would also help. Future research could also use data from several years and repeat the split to check whether the differences between the models are robust.

References

US Department of Education (USDE). (2026). *Equity in Athletics Data Analysis*. https://ope.ed.gov/athletics/#/, retrieved 29.10.2026

US Department of Education (USDE). (2025). *EADA survey: Total revenues screen instructions*. https://surveys.ope.ed.gov/athletics2025/wwwroot/documents/inst_1.pdf, retrieved 29.10.2026

Appendix

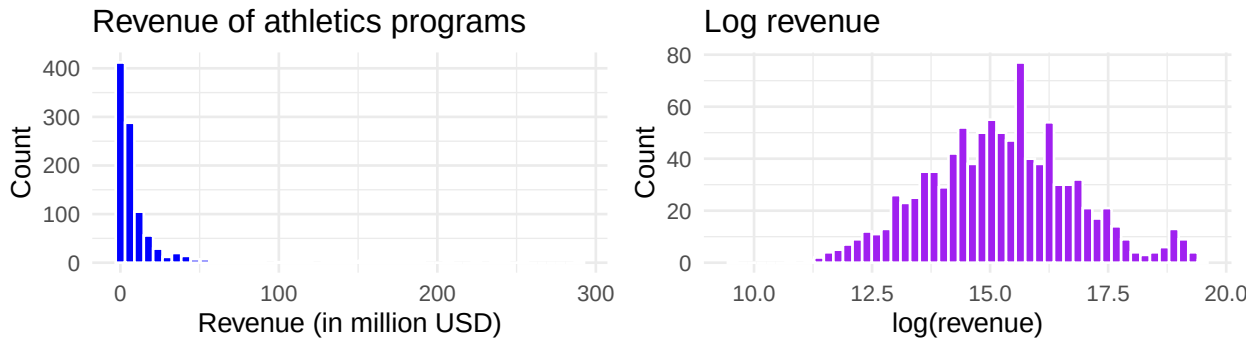


Figure 1: Distribution of revenue in million USD (left) and log revenue (right). Revenue is strongly right-skewed, log revenue is roughly symmetric.

Table 1: Eigenvalues, VAF (in %), upper bound of the permutation test and bootstrap confidence intervals of the eigenvalues for the first six components.

Component	Eigenvalue	VAF	Cumulative VAF	Permutation 97.5%	Bootstrap 95% CI
1	13.80	49.29	49.29	1.42	[13.28, 14.33]
2	2.74	9.77	59.07	1.35	[2.50, 3.09]
3	1.50	5.36	64.43	1.31	[1.39, 1.76]
4	1.23	4.40	68.83	1.27	[1.14, 1.49]
5	1.12	3.98	72.82	1.24	[1.04, 1.23]
6	1.02	3.65	76.47	1.21	[0.94, 1.10]

Table 2: Component loadings (correlations between variables and components C1 to C3) and fit per variable.

Variable	C1	C2	C3	Fit	Variable	C1	C2	C3	Fit
ugrad men	0.58	0.53	0.33	0.72	partic track women	0.78	0.15	0.24	0.70
ugrad women	0.53	0.57	0.36	0.73	partic volleyball women	0.63	-0.18	-0.20	0.47
athletes men	0.89	-0.15	-0.07	0.81	partic other men	0.75	-0.31	0.21	0.70
athletes women	0.94	-0.01	-0.02	0.88	partic other women	0.82	-0.08	0.27	0.75
teams men	0.86	-0.39	0.07	0.89	hc men	0.86	-0.38	0.05	0.89
teams women	0.92	-0.20	0.07	0.89	hc women	0.92	-0.20	0.05	0.90
partic football	0.59	0.09	-0.04	0.36	ac men	0.89	-0.03	0.07	0.79
partic bball men	0.48	-0.29	-0.48	0.55	ac women	0.87	0.03	0.14	0.79
partic bball women	0.60	-0.14	-0.39	0.53	hc salary men	0.64	0.52	-0.14	0.70
partic baseball	0.45	-0.18	-0.37	0.37	hc salary women	0.67	0.41	-0.24	0.68
partic softball	0.54	-0.10	-0.30	0.39	ac salary men	0.63	0.49	-0.26	0.70
partic soccer men	0.40	-0.51	0.14	0.44	ac salary women	0.63	0.46	-0.29	0.69
partic soccer women	0.70	-0.14	0.19	0.54	hc fulltime share	0.50	0.44	-0.21	0.49
partic track men	0.74	0.00	0.22	0.59	hc female share	0.13	0.25	0.11	0.09

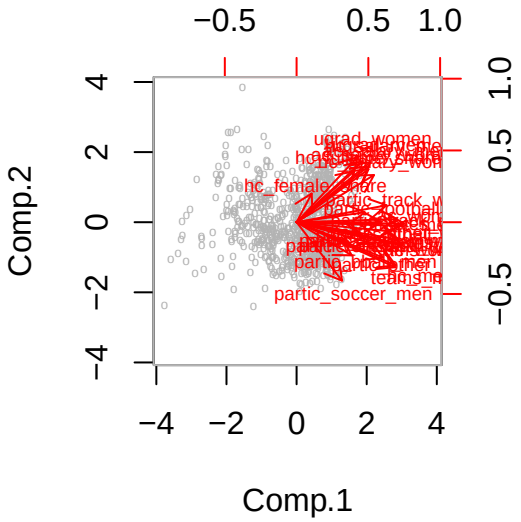


Figure 2: Biplot of components 1 and 2. Points are institutions, arrows are variables.

Table 3: Predictive performance on the test set.

Model	RMSE (log)	R^2 (log)	RMSE (million USD)
PCR	0.425	0.916	9.059
Ridge ($\lambda_{\min}$)	0.447	0.907	8.781
Ridge (λ_{1SE})	0.456	0.903	9.378

Table 4: Test RMSE on the log scale per revenue group, each containing 10% of the test institutions (1 = lowest, 10 = highest revenue).

	1	2	3	4	5	6	7	8	9	10
PCR	0.57	0.72	0.40	0.31	0.32	0.26	0.21	0.33	0.35	0.52
Ridge ($\lambda_{\min}$)	0.71	0.73	0.35	0.31	0.31	0.30	0.24	0.37	0.32	0.51

Table 5: Predicted revenue (in million USD) of the 10 institutions.

ID	Classification	PCR	Ridge
1	NCAA Division I-FBS	115.2	142.2
2	NCAA Division I-FCS	17.7	18.5
3	NCAA Division I without football	11.6	10.1
4	NCAA Division II with football	10.5	9.3
5	NCAA Division II without football	4.7	3.6
6	NCAA Division III with football	3.5	3.2
7	NCAA Division III without football	3.7	4.1
8	NAIA Division I	1.7	1.7
9	NJCAA Division I	2.2	2.7
10	CCCAA	4.4	4.7

Code

```
library(tidyverse); library(glmnet); library(boot); library(gridExtra); library(caret)

permtestPCA <- function(X, nTests = 100, alpha = 0.05, ...){ # permutation test function from lecture 5
  n <- nrow(X); m <- ncol(X); X <- scale(X)
  eigs.X <- prcomp(X)$sdev^2; eigs.Xperm <- matrix(0, m, nTests); Xperm <- X
  for (i in 1:nTests){
    for (j in 2:m) Xperm[, j] <- X[sort(runif(n), index.return = TRUE)$ix, j] # random permutation of the rows per column
    eigs.Xperm[, i] <- prcomp(Xperm)$sdev^2}
  perc.alpha <- t(apply(eigs.Xperm, 1, quantile, c(alpha/2, 1 - alpha/2)))
  plot(1:m, eigs.X, type = "b", col = "red", main = "Permutation test PCA", xlab = "Component", ylab = "Eigenvalue", ...)
  matlines(1:m, perc.alpha, type = "b", col = "blue", lty = 1)
  perc.alpha}
f2 <- function(x, d = 2) formatC(x, format = "f", digits = d)    # rounds numbers

athletics  <- read.csv("data_783996yw.csv"); prediction <- read.csv("predictions-2.csv")
dim(athletics); summary(athletics$revenue)                       # size and distribution of revenue
colSums(is.na(athletics))[colSums(is.na(athletics)) > 0]         # missing values only in the coach variables
table(no_men_teams = athletics$teams_men == 0, hc_missing = is.na(athletics$hc_men)) # no men's team -> no men's hc
min(athletics$ac_men, na.rm = TRUE)                              # ac never 0
sum(!is.na(athletics$ac_men) & is.na(athletics$ac_salary_men))  # acs but no salary (volunteers?)
table(athletics$classification); table(athletics$sector)         # some levels are very small
revenue <- ggplot(athletics, aes(revenue / 1e6)) + geom_histogram(bins = 50, fill = "blue", color = "white") +
  labs(title = "Revenue of athletics programs", x = "Revenue (in million USD)", y = "Count") + theme_minimal()
logrevenue <- ggplot(athletics, aes(log(revenue))) + geom_histogram(bins = 50, fill = "purple", color = "white") +
  labs(title = "Log revenue", x = "log(revenue)", y = "Count") + theme_minimal()
grid.arrange(revenue, logrevenue, ncol = 2)                      # appendix figure 1: very skewed, the log is more symmetric

count_vars  <- c("ugrad_men", "ugrad_women", "athletes_men", "athletes_women", "teams_men", "teams_women",
                 grep("^partic_", names(athletics), value = TRUE), "hc_men", "hc_women", "ac_men", "ac_women") # logged
salary_vars <- c("hc_salary_men", "hc_salary_women", "ac_salary_men", "ac_salary_women")                  # logged
share_vars  <- c("hc_fulltime_share", "hc_female_share")                                                 # 0 to 1, not logged
num_vars    <- c(count_vars, salary_vars, share_vars)                                                    # 28 numerical predictors
fixed_coaches <- function(x) {                                   # runs once for men's and once for women's variables
  for (s in c("men", "women")) {
    hc <- paste0("hc_", s); ac <- paste0("ac_", s); hs <- paste0("hc_salary_", s); as <- paste0("ac_salary_", s)
    no_hc <- is.na(x[[hc]]); no_ac <- is.na(x[[ac]])             # institutions without hcs / acs
    x[[hc]][no_hc] <- 0; x[[ac]][no_ac] <- 0                     # no coaches -> 0
    x[[hs]][no_hc & is.na(x[[hs]])] <- 0; x[[as]][no_ac & is.na(x[[as]])] <- 0 # coaches without salary stay na
  }
  x}
logged_vars <- function(x) { for (v in c(count_vars, salary_vars)) x[[v]] <- log1p(x[[v]]); x } # log(x + 1) as x can be 0
athletics_new  <- fixed_coaches(athletics); prediction_new <- fixed_coaches(prediction)
athletics_new$log_revenue <- log(athletics_new$revenue)          # dependent variable
athletics_new$classification <- factor(athletics_new$classification); athletics_new$sector <- factor(athletics_new$sector)
prediction_new$classification <- factor(prediction_new$classification, levels = levels(athletics_new$classification)) # same levels -> same dummies
prediction_new$sector <- factor(prediction_new$sector, levels = levels(athletics_new$sector))
athletics_new <- logged_vars(athletics_new); prediction_new <- logged_vars(prediction_new)

set.seed(123)
trainRowNumbers <- createDataPartition(athletics_new$log_revenue, p = 0.8, list = FALSE)
train_data <- athletics_new[trainRowNumbers, ]; test_data <- athletics_new[-trainRowNumbers, ]
train_medians <- sapply(train_data[num_vars], median, na.rm = TRUE) # training data only
impute <- function(x) { for (v in num_vars) x[[v]][is.na(x[[v]])] <- train_medians[v]; x }
train_data <- impute(train_data); test_data <- impute(test_data); prediction_new <- impute(prediction_new)
round(sort(sapply(train_data[num_vars], cor, y = train_data$log_revenue), decreasing = TRUE), 2) # correlations with log revenue

X_pca <- train_data[, num_vars]; X_pca_test <- test_data[, num_vars]; p <- ncol(X_pca)
fitpca <- princomp(X_pca, cor = TRUE, scores = TRUE)             # correlation matrix
eigenvalues <- fitpca$sdev^2; vaf <- eigenvalues / p; cum_vaf <- cumsum(vaf)
screeplot(fitpca, npcs = p, type = "lines", main = "Scree plot"); abline(h = 1, col = "red", lty = 2) # elbow and kaiser line
k_kaiser <- sum(eigenvalues > 1)                                 # kaiser: eigenvalue > 1
k_vaf80  <- which(cum_vaf >= 0.80)[1]                            # 80% cumulative vaf
set.seed(123); perm_range <- permtestPCA(X_pca, nTests = 1000)  # permutation test
k_perm <- which(eigenvalues <= perm_range[, 2])[1] - 1           # keep pcs above the 97.5% interval bound
my_boot_pca <- function(x, ind) princomp(x[ind, ], cor = TRUE)$sdev^2 # pca on resampled rows -> eigenvalues
set.seed(123); eigs.boot <- boot(data = X_pca, statistic = my_boot_pca, R = 1000)$t
kaiser_boot <- data.frame(PC = 1:p, Eigenvalue = eigenvalues, Lower = apply(eigs.boot, 2, quantile, 0.025),
                          Upper = apply(eigs.boot, 2, quantile, 0.975))
k_kaiser_boot <- which(kaiser_boot$Lower <= 1)[1] - 1            # bootstrapped kaiser, whole confidence interval above 1
round(data.frame(kaiser_boot[1:6, 1:2], VAF = 100 * vaf[1:6], Cum_VAF = 100 * cum_vaf[1:6], Perm_97.5 = perm_range[1:6, 2], kaiser_boot[1:6, 3:4]),
       2) # appendix table 1
data.frame(Method = c("Kaiser", "80% VAF", "Permutation test", "Bootstrapped Kaiser"),
           PCs = c(k_kaiser, k_vaf80, k_perm, k_kaiser_boot))
n_pc <- k_perm                                                   # chosen: permutation test
vaf.boot   <- rowSums(eigs.boot[, 1:n_pc, drop = FALSE]) / p    # bootstrapped vaf of the chosen pcs
perc.alpha <- quantile(vaf.boot, c(0.025, 0.975)); round(100 * c(cum_vaf[n_pc], perc.alpha), 1)
corpca  <- cor(X_pca, fitpca$scores)[, 1:n_pc]                   # loadings
fit_var <- cbind(corpca, Fit = rowSums(corpca^2)); round(fit_var, 2) # fit per variable (appendix table 2)
biplot(fitpca, pc.biplot = TRUE, scale = 1, choices = 1:2, col = c("grey", "red"), asp = 1,
       cex = c(0.4, 0.6), xlabs = rep("o", nrow(X_pca)))      # appendix figure 2: points = institutions, arrows = variables

train_scores <- data.frame(fitpca$scores[, 1:n_pc], classification = train_data$classification,
                           sector = train_data$sector, log_revenue = train_data$log_revenue)
test_scores  <- data.frame(predict(fitpca, newdata = X_pca_test)[, 1:n_pc], # test projected on the training pca
                           classification = test_data$classification, sector = test_data$sector)
fit_pcr <- lm(log_revenue ~ ., data = train_scores); summary(fit_pcr)

form_x <- as.formula(paste("~", paste(c(num_vars, "classification", "sector"), collapse = " + ")))
y <- train_data$log_revenue
X <- model.matrix(form_x, train_data)[, -1]                      # dummies, intercept removed
X_test <- model.matrix(form_x, test_data)[, -1]; X_new <- model.matrix(form_x, prediction_new)[, -1]
x_mean <- colMeans(X[, num_vars]); x_sd <- apply(X[, num_vars], 2, sd) # z-scores with training means and sds
X[, num_vars] <- scale(X[, num_vars], x_mean, x_sd); X_test[, num_vars] <- scale(X_test[, num_vars], x_mean, x_sd)
X_new[, num_vars] <- scale(X_new[, num_vars], x_mean, x_sd)
grid <- 10^seq(-5, 2, length.out = 100)                          # grid of lambdas
plot(glmnet(X, y, alpha = 0, lambda = grid, standardize = FALSE), xvar = "lambda") # coefficient paths
set.seed(123); result.ridge.cv <- cv.glmnet(X, y, alpha = 0, lambda = grid, nfolds = 10, standardize = FALSE)
c(result.ridge.cv$lambda.min, result.ridge.cv$lambda.1se)        # lambda.min and 1se rule
result.ridge.min <- glmnet(X, y, alpha = 0, lambda = result.ridge.cv$lambda.min, standardize = FALSE)
```

```
result.ridge.1se <- glmnet(X, y, alpha = 0, lambda = result.ridge.cv$lambda.1se, standardize = FALSE)
round(cbind(result.ridge.min$beta, result.ridge.1se$beta), 3)   # coefficients per sd for numerical predictors

rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))
y_test <- test_data$log_revenue; rev_test <- test_data$revenue
pred_log <- list("PCR" = predict(fit_pcr, newdata = test_scores),
                 "Ridge lambda.min" = as.numeric(predict(result.ridge.min, newx = X_test)),
                 "Ridge lambda.1se" = as.numeric(predict(result.ridge.1se, newx = X_test)))
eval_tab <- do.call(rbind, lapply(names(pred_log), function(m) { pl <- pred_log[[m]]
  data.frame(Model = m, RMSE_log = rmse(y_test, pl), R2_log = 1 - sum((y_test - pl)^2) / sum((y_test - mean(y_test))^2),
             RMSE_rev_M = rmse(rev_test, exp(pl)) / 1e6) }))   # rmse in million usd after exp()
eval_tab                                                      # appendix table 3
err_by_class <- data.frame(classification = test_data$classification,   # where do the usd errors come from?
                           sq_err_rev = (rev_test - exp(pred_log[["Ridge lambda.min"]]))^2) %>%
  group_by(classification) %>% summarise(n = n(), share_of_squared_err = sum(sq_err_rev)) %>%
  mutate(share_of_squared_err = share_of_squared_err / sum(share_of_squared_err)) %>% arrange(desc(share_of_squared_err))
err_by_class                                                  # fbs schools cause most of the usd error
dec <- cut(rank(y_test), 10, labels = 1:10)                   # 10 equally sized revenue groups of the test set (1 = lowest)
rmse_dec <- sapply(pred_log, function(pl) tapply((y_test - pl)^2, dec, function(e) sqrt(mean(e)))); round(rmse_dec, 2) # rmse per revenue group (
      appendix table 4)
round(sapply(pred_log, function(pl) tapply(pl - y_test, dec, mean)), 2) # mean error per revenue group, < 0 = underpredicted

new_scores <- data.frame(predict(fitpca, newdata = prediction_new[, num_vars])[, 1:n_pc],
                         classification = prediction_new$classification, sector = prediction_new$sector)
pred_out <- data.frame(id = prediction_new$id, classification = prediction_new$classification,
                       rev_PCR_M = exp(predict(fit_pcr, newdata = new_scores)) / 1e6,          # back to usd with exp()
                       rev_Ridge_M = exp(as.numeric(predict(result.ridge.min, newx = X_new))) / 1e6)
pred_out                                                      # appendix table 5
```